

SANJEEVNI

48-Hour Build Guide & Role Playbook

Smart India Hackathon 2026 — Problem Statement 26181

Team BrajCoders | Team ID: 1111

What Sanjeevni Is

A smart undergarment with embedded ECG, motion, temperature/humidity and air-quality sensors, hubbed by an ESP32 that streams data over Bluetooth Low Energy to a companion phone app. The app runs on-device inference to flag cardiac irregularities, heat stress, respiratory risk and falls in real time, fully offline, with SMS-based SOS when no internet is available. It pulls weather/AQI context from open public APIs when online and degrades gracefully to on-device sensing when it isn't.

Golden Rule for the Next 48 Hours

Build the **thinnest possible version of every layer** before you make **any single layer deep**. A judge sees a phone screen showing live vitals and a heat alert firing — not how clever your filter math is underneath it.

How to use this document: each teammate should read the Common Ground section once, then jump straight to their own role section. Paste your section into an AI assistant, tell it your role, and use it to get unstuck – the document is written to be self-contained per person.

Contents

1	Common Ground – Read This First (Everyone)	2
1.1	The Problem We Are Actually Solving	2
1.2	The System, End to End	2
1.3	Team Map	2
1.4	The One Priority Rule Everyone Follows	2
2	Hardware Person – Sensor Placement, Wiring & Acquisition	3
2.1	Sensor Bill of Materials	3
2.2	Sensor Placement – What the Research Actually Says	4
2.3	ESP32 Responsibilities (Keep It Dumb)	4
2.4	First 4 Hours – Your Concrete Checklist	5
3	AI / Edge AI Person – Signal Processing, Models & Personalization	6
3.1	The Signal Chain (Do This in Order)	6
3.2	The Four Detection Models – Realistic 48-Hour Versions	7
3.3	Personalization – What’s Actually Buildable	7
3.4	Fusing In External Context (Open APIs)	7
3.5	Your First 4 Hours	8
4	Mobile App Team (2 People) – Feature Ownership & Priority	9
4.1	Suggested Split	9
4.2	Prioritized Feature List	9
4.3	Practical Notes	10
5	Database / Data Layer Person	11
5.1	Recommended Setup	11
5.2	Your First 4 Hours	11
6	Frontend Design + PPT/Demo Person	12
6.1	What’s Already Strong in Your Existing Deck	12
6.2	P1 – What to Build/Polish First	12
6.3	Your First 4 Hours	12
7	48-Hour Master Timeline	13

1 Common Ground -- Read This First (Everyone)

1.1 The Problem We Are Actually Solving

India's climate disasters (heat waves, floods, pollution spikes) hit vulnerable people — the elderly, outdoor workers, rural populations — hardest, and precisely when internet and healthcare access break down. Existing wearables either lack real ECG (fitness bands), require constant connectivity (Apple Watch, cloud health apps), or send raw health data off-device (privacy risk). Sanjeevni's bet is: **do the health reasoning on the device itself**, so it keeps working when the network doesn't.

1.2 The System, End to End

Layer	What it does
Tier 1 – Sensing	ECG electrodes, PPG (MAX30102), DHT22/BME280 (temp/humidity), MQ135 (air quality), MPU6050 (motion) embedded in garment, wired into an ESP32
Tier 1.5 – ESP32 hub	Samples sensors, does lightweight filtering, packages features, sends over BLE
Tier 2 – Phone AI	Receives BLE stream, runs on-device rules/models (fall, heart, respiratory, heat), fuses with open weather/AQI API data, decides alerts
Tier 3 – App UI	Dashboard, alerts, SOS, trends, personalization
Data layer	On-device storage (time-series-ish), local history, no mandatory cloud round-trip

1.3 Team Map

Role	People
Hardware + On-device signal pipeline	1 person (paired with AI person)
AI / Edge AI (models, filtering, personalization)	1 person (paired with hardware person)
Mobile App – Core (BLE, data ingestion, dashboard)	1 person
Mobile App – Alerts & Integrations (SOS, weather/AQI API, notifications)	1 person
Database / Data Layer	1 person
Frontend Design + PPT/Demo	1 person

1.4 The One Priority Rule Everyone Follows

Every feature in this document is tagged:

- **[P1 – MUST HAVE]** – the demo does not work without this. Build it in the first 18 hours.
- **[P2 – HIGH VALUE]** – makes the demo convincing and matches judging criteria directly.
- **[P3 – IF TIME PERMITS]** – nice, do it only after all P1/P2 across *all* roles are done.
- **[STRETCH – CUT FIRST]** – talk about it in the PPT as "future scope," do not attempt to build it.

Do Not Do This

Do not let any one person "perfect" their layer while other layers are still at zero. A fully-working ECG filter with no app to show it on is worth nothing to a judge. Integrate early and often – get a fake/dummy data stream flowing end-to-end on **hour 4**, before real sensor data is even ready.

2 Hardware Person -- Sensor Placement, Wiring & Acquisition

Your Mission

Build a garment that reliably gets ECG, motion, temperature/humidity and air-quality signal off a human body onto an ESP32, and get that data moving over BLE. You are not responsible for interpreting the data – that is the AI person’s job. Your job is **clean signal acquisition and transport**. You work in a tight loop with the AI person since they consume your raw stream directly.

2.1 Sensor Bill of Materials

Sensor	Measures	Priority
ECG front-end (AD8232 or BioAmp)	Heart electrical rhythm, HRV, arrhythmia cues	[P1 – MUST HAVE]
MAX30102 (PPG)	Heart rate, SpO2 (secondary, chest-PPG is noisy)	[P2 – HIGH VALUE]
MPU6050 (accel + gyro)	Motion, fall detection, activity	[P1 – MUST HAVE]
DHT22 or BME280	Ambient temperature + humidity	[P1 – MUST HAVE]
MQ135 / PM2.5 module	Air quality proxy	[P2 – HIGH VALUE]

2.2 Sensor Placement -- What the Research Actually Says

Sensor	Placement & Why
ECG electrodes	Use a <i>modified lead-II / CM5</i> -style placement rather than clinical limb leads – this is the standard approach for wearable chest-strap ECG because it maximizes the QRS amplitude while tolerating trunk motion far better than limb leads. Place two sensing electrodes: one near the right side below the clavicle (upper right chest) and one near the lower left ribcage (apex area), with a ground/reference electrode on the lower right ribcage or sternum. This mimics standard Polar/chest-strap HR monitor geometry, which is well validated in wearable ECG literature. Keep electrode spacing as wide as the garment allows – wider spacing = stronger signal, but do not cross the sternum awkwardly or fit will suffer.
PPG (MAX30102)	Chest-based PPG is inherently noisier than fingertip/wrist PPG because chest perfusion is lower and motion artifact is higher. Place it flat against skin near the sternum, snug fit is critical (any air gap ruins the reading). Treat this as a <i>secondary, lower-confidence</i> HR/SpO2 source – your primary heart-rate value should come from the ECG R-peak intervals, not PPG. Be upfront with your team that PPG SpO2 accuracy on chest placement is a known weak point (this is fine to state honestly in the PPT).
MPU6050 (motion)	Mount centrally near the sternum/chest, not on a limb. Chest-centered accelerometers give the best trunk-based fall signature (a fall shows as a fast free-fall drop in accel magnitude, followed by a sharp impact spike, followed by prolonged stillness) and this placement doubles as your best proxy for general activity/step-like motion since it moves with the torso, not swinging limbs.
DHT22 / BME280 (ambient)	This is the one sensor that must <i>not</i> be pressed against skin, or you'll measure body heat, not ambient conditions. Mount it on an outward-facing point of the garment (e.g., a small vented pocket near the collar or shoulder) so it samples the air around the person, not their skin temperature.
MQ135 / PM2.5	Mount near the collar/neckline – this approximates the person's actual breathing zone, which is what you want for a respiratory-risk-relevant air quality reading, and it stays clear of body heat that would bias a chest-mounted reading.

Reality Check

Dry-electrode ECG signal degrades with sweat and heavy movement. Do not try to solve this fully in 48 hours. Mitigate with (a) firm, even garment compression at the electrode sites, (b) accepting some noise and letting the AI layer flag/discard noisy windows rather than trying to eliminate noise at the hardware level, (c) telling judges this is an acknowledged limitation with a clear future-scope answer (gel electrodes / better textile contacts). Being honest about this scores better than pretending it is solved.

2.3 ESP32 Responsibilities (Keep It Dumb)

P1 -- Core Acquisition Loop

1. Sample ECG at 250–360 Hz (this is the standard range used in ECG research datasets like MIT-BIH, and is enough for R-peak/HRV detection without overwhelming BLE bandwidth).
2. Sample MPU6050 at 50–100 Hz (fast enough to catch a fall's impact spike).
3. Sample DHT22/BME280 and MQ135 at 1 sample every 5–10 seconds – these change slowly, no need to sample fast, and it saves power + bandwidth.
4. Do only *trivial* on-device filtering here: a simple moving-average or a basic digital notch at 50 Hz

(India's mains frequency) to knock out powerline hum before it ever leaves the chip. Do not attempt real feature extraction on the ESP32 – leave that to the phone. Time spent making the ESP32 "smart" is time not spent on getting the demo end-to-end.

5. Package a compact struct/JSON packet and push it over BLE (GATT characteristic, notify-on-change) every 1 second or so at minimum for the fast sensors; slow sensors can piggyback less frequently.

P2

- Buffer a few seconds of data locally on the ESP32 if BLE briefly disconnects, so you don't lose data on every minor connection drop.
- Add a status LED / buzzer driven by ESP32 GPIO for a simple physical "connected/alert" indicator – cheap, and looks great in a live demo.

Haptic feedback motor, onboard actuator control, full local SOS logic on the ESP32 itself. Your PPT already shows these as part of the long-term vision – do not build them now.

2.4 First 4 Hours -- Your Concrete Checklist

1. Get *any* single sensor (start with MPU6050, it's the easiest to verify) reading values over serial monitor.
2. Get ESP32 broadcasting a BLE GATT service with a dummy/fake value, and confirm the mobile app person can actually connect and read it. This unblocks the whole team and should happen before real sensor integration is finished.
3. Only after that handshake works, wire in ECG, then ambient sensors, in that order.

3 AI / Edge AI Person -- Signal Processing, Models & Personalization

Your Mission

Turn the raw sensor stream coming off the ESP32 into decisions a human can act on: "this heart rhythm looks abnormal," "heat stress risk is high," "this looks like a fall." In 48 hours, "AI" mostly means **well-designed signal processing plus transparent rule/statistical logic**, not trained neural networks – and that is the right, honest choice, not a compromise. This is the section your team should spend the most relative time on, since it is your product's central differentiator.

Where Inference Actually Runs -- Be Honest About This

The PPT says "on-device AI" and "TensorFlow Lite" – that claim should refer to inference happening **on the phone**, not the ESP32. The ESP32 (typically 240MHz, no dedicated NN accelerator) cannot realistically run a TFLite model in your timeframe, and attempting it will burn a day you don't have. The correct, still fully truthful architecture is: **ESP32 acquires and lightly filters** → **phone runs the actual inference/rules via TFLite or plain on-device code**. This matches "on-device" (nothing leaves the phone to a cloud server) and is what "edge AI" means in almost every real wearable product today.

3.1 The Signal Chain (Do This in Order)

P1 -- ECG Preprocessing Pipeline

1. **Notch filter at 50 Hz** to remove Indian AC powerline interference – this is close to mandatory for any ECG signal collected indoors in India, and it's a small, well-known filter (IIR notch, narrow bandwidth around 50 Hz).
2. **Bandpass filter roughly 0.5–40 Hz** to remove baseline wander (low frequency drift from breathing/movement) and high-frequency EMG/muscle noise, while preserving the actual QRS complex shape.
3. **R-peak detection** using a simple threshold-and-refractory-period approach (a lightweight Pan-Tompkins-style method is the standard, well-documented choice: differentiate → square → moving-window integrate → adaptive threshold). This gives you beat-to-beat R-R intervals.
4. **Heart rate** = $60 / (\text{average R-R interval in seconds})$ over a rolling window.
5. **HRV / rhythm irregularity** = variance or standard deviation of R-R intervals over the last N beats. A sudden spike in this variance, or R-R intervals far outside the person's own recent baseline, is your "possible arrhythmia" flag – *not* a diagnosis, frame it as a risk flag only.
6. **Motion-artifact gating**: use the MPU6050 stream to detect when the person is moving heavily, and suppress/down-weight ECG-based alerts during those windows since motion is the single biggest source of false alarms in wearable ECG.

3.2 The Four Detection Models -- Realistic 48-Hour Versions

Model	48-hour realistic approach	Priority
Heart Health	Rolling baseline HR + R-R variance per user; flag when current reading deviates beyond a threshold (e.g., 2 standard deviations) from their own recent baseline, gated by motion state	[P1 – MUST HAVE]
Heat Stress	Compute a heat index from ambient temperature + humidity using the standard heat-index formula (Rothfusz regression, the same approach meteorological services use), combine with elevated HR to raise confidence	[P1 – MUST HAVE]
Fall Detection	Accelerometer magnitude pattern: near-zero-g free-fall dip → sharp impact spike → sustained stillness after impact. This threshold-sequence approach is the standard lightweight fall-detection method used in real wearables	[P2 – HIGH VALUE]
Respiratory Risk	Combine ambient AQI/MQ135 reading with any available breathing-rate proxy (chest motion cycle from accelerometer, if time allows) or simply AQI thresholds + person's risk profile (elderly/chronic condition flag they set in-app)	[P2 – HIGH VALUE]

A trained CNN arrhythmia classifier on MIT-BIH data (as your PPT lists under future scope) needs a labeled dataset, training time, and quantization/conversion to TFLite – this is a multi-week effort done properly, not a 48-hour one. State it as future scope exactly as your deck already does; do not attempt a rushed version, since a broken "AI model" is worse for the demo than an honest, well-explained rule engine.

3.3 Personalization -- What's Actually Buildable

P2

Maintain a simple rolling per-user baseline: for each metric (resting HR, typical R-R variance, typical activity level), keep a moving average over the last 3–7 days of data (or last N sessions if you don't have days). Compare live readings against *that person's* baseline rather than a fixed population threshold. This is honest, explainable "personalization" – it is not machine learning in the training sense, and that's fine to say plainly: it's adaptive thresholding, and it's exactly the right amount of sophistication for this timeframe.

3.4 Fusing In External Context (Open APIs)

P1

When the phone has connectivity, pull local weather + AQI from a free public API (e.g., OpenWeatherMap or the Indian government's AQI data sources) using the phone's GPS location, and use that alongside/as a fallback for the on-garment MQ135 and ambient temperature sensors. This lets your heat-stress and respiratory-risk models work even before the hardware sensors are fully calibrated, and gives you a fallback if a hardware sensor is flaky on demo day.

P2

When offline, fall back entirely to on-garment ambient sensor readings for heat/AQI context – this is the actual point of your product’s “works without internet” claim, so make sure this fallback path is demoed, not just the online path.

3.5 Your First 4 Hours

1. Get a fake/simulated ECG waveform (a public sample, or a simple synthetic sine + noise) flowing through your notch → bandpass → R-peak pipeline in a notebook/script, completely independent of hardware.
2. Confirm HR and HRV numbers come out sane on that fake data.
3. As soon as the hardware person has *any* real ECG samples coming off the ESP32 (even noisy), swap your fake data source for the real one – don’t wait for “clean” hardware data before integrating.

4 Mobile App Team (2 People) -- Feature Ownership & Priority

Your Mission

Two of you own the entire phone app: receiving the BLE stream, running/consuming the AI person's logic, showing a dashboard, and handling alerts + SOS. Split the work by **surface** so you don't collide on the same files: one person owns the data/connectivity core, the other owns alerts, integrations, and outward-facing UI polish.

4.1 Suggested Split

Person A – Core & Data

BLE connection to ESP32, receiving/parsing live sensor packets, wiring received data into local storage, live vitals dashbo

4.2 Prioritized Feature List

P1 -- Both People -- First 12 Hours

- BLE scan + connect to the ESP32's GATT service, and reliably re-parse its data packets (Person A)
- A single "Live Vitals" screen showing HR, SpO2, ambient temp, AQI, and a basic status indicator (Normal / Caution / Risk) – this is the single most important screen in the whole product for a live demo

(Person A)

- Local storage write-path: every incoming reading gets saved (hand off to Database person's schema) (Person A)
- Basic alert banner/notification when the AI layer flags a risk (e.g., "High Heat Alert – 42°C, stay hydrated") (Person B)
- Weather/AQI API call using phone GPS, displayed alongside on-device readings (Person B)

P2 -- Next 12 Hours

- Trend view / simple history graph for HR and heat-risk over the last few hours (Person A)
- SOS button: on tap, send an SMS (using the phone's native SMS capability, no internet required) to a pre-set emergency contact with a canned message + last known location if permission granted (Person B)
- "Disconnected / working offline" indicator so it's visually obvious the app still functions without internet during the demo (Person B)
- Fall-detected alert with a countdown-to-auto-SOS behavior (cancel within N seconds if it was a false alarm) (Person B)

P3 -- Only If P1/P2 Done Across Whole Team

- Risk score visualizations per category (heat/AQI/flood/storm) matching the PPT's dashboard mockup styling
- Recent alerts/updates feed / history log screen
- Basic user profile with vulnerability flags (elderly, chronic condition) that feed into the AI person's risk thresholds

Caregiver/family dashboard view, multi-user accounts, regional-language voice alerts – explicitly future scope per your own deck. Do not touch these in 48 hours.

4.3 Practical Notes

- **Do not wait for real hardware** to start the BLE integration – ask the hardware person for a dummy/fake BLE service emitting hardcoded values on hour 1–2, and build against that. Swap in the real stream later; the app-side code barely changes.
- **Do not wait for the AI person's final logic** to build the alert UI – build the alert banner against a hardcoded/mock "risk level" value first, then wire the real AI output in once it exists.
- Keep the dashboard visually close to what's already in the PPT mockup (the frontend person owns final polish, but structurally match it early so integration is trivial later).

5 Database / Data Layer Person

Your Mission

Decide where every reading, alert, and user setting lives, and make sure the mobile team can write to and read from it without friction. Given this product's entire pitch is privacy-preserving and offline-first, your default should be an **on-device** data layer, not a cloud database.

Reality Check on InfluxDB / Cloud Options

A dedicated time-series database like InfluxDB is built for server-side, networked deployments – it doesn't meaningfully run "on-device" inside a phone app in the way this product needs, and standing up a cloud instance directly contradicts the "nothing sent to the cloud" claim that is your product's core differentiator versus the cloud-based competitors in your own comparison slide. Unless the team explicitly wants an *optional* caregiver-sync feature (P3 at best), skip cloud databases entirely and keep everything local.

5.1 Recommended Setup

P1

- **SQLite** (via Room, since the app is likely React Native/Android – coordinate the exact binding library with the mobile team) as the single on-device store for all time-series-like readings (HR, SpO2, temp, AQI, motion events) and for alerts/events history.
- Design tables around: `readings` (timestamp, sensor_type, value, source=[garment/api]), `alerts` (timestamp, category, severity, message, acknowledged), `user_profile` (baseline values, vulnerability flags, emergency contact).
- Index on timestamp for fast "last N minutes" queries – this is what the live dashboard and trend graphs will hit constantly.

P2

- A simple rolling-baseline table that the AI person's personalization logic reads/writes (per-metric moving average, updated on a schedule) – this is the concrete data-layer piece the AI person's P2 feature depends on, so prioritize this once P1 is done.
- Basic data retention/pruning (e.g., keep raw readings for 7 days, keep daily summaries indefinitely) so the local DB doesn't bloat during long demo/testing sessions.

Optional cloud sync for a caregiver dashboard, multi-device sync, InfluxDB/Firebase cloud backend. Mention in PPT as a scalability future step (your deck's "Firebase" tech stack entry is fine to keep as a stated future option) – do not build during the hackathon.

5.2 Your First 4 Hours

1. Agree the exact schema (field names, types) with both mobile app people *before* anyone writes code against it – this single conversation prevents the most common last-minute integration breakage.
2. Stand up the SQLite tables with a few rows of dummy data so Person A (mobile) can build the dashboard's read-path immediately, in parallel with real data flowing in.

6 Frontend Design + PPT/Demo Person

Your Mission

You own two things: the app's visual polish (working closely with both mobile devs so their screens actually match the PPT's mockup aesthetic) and the pitch itself – slides plus, critically, the live demo script. In a hackathon, a confident, well-paced demo usually matters more than any single slide.

6.1 What's Already Strong in Your Existing Deck

- The problem framing with concrete stats (554 heatwave days, 75% outdoor workforce, rural vs urban internet gap) is compelling and specific – keep it.
- The competitor comparison table (fitness bands vs premium wearables vs cloud apps) is a strong, clear differentiation device – keep it front and center.
- The "Future Scope & Limitations" slide being explicit and honest is a real strength; judges respond well to teams that know their own limits.

6.2 P1 -- What to Build/Polish First

P1

- A tight, rehearsed **live demo script**: exact sequence of taps/actions that shows (a) live vitals updating from the actual garment, (b) a heat or fall alert firing, (c) the offline SOS path, in under 90 seconds. Write this script down and rehearse it with whoever is presenting – this matters more than any slide.
- Make sure the actual app UI matches your mockup's visual language (colors, card layout, iconography) closely enough that switching between slides and the live app doesn't feel jarring.

P2

- Update the technical architecture slide to reflect the honest split of where processing happens (ESP32 = acquisition/filtering, phone = actual AI/decisions) – this avoids a judge catching an inconsistency between the slide and a technical Q&A answer.
- Prepare 2–3 backup slides/screenshots of the app in case a live BLE demo fails on stage (a very common hackathon failure mode) – have a recorded video or screenshots as fallback.

Backup Plan Is Not Optional

Live hardware demos fail at hackathons more often than they succeed, usually due to BLE pairing issues under stage conditions (RF interference from other teams' devices). Record a clean video of the full flow working on hour 40ish, regardless of confidence in the live version, and have it ready to play if needed.

6.3 Your First 4 Hours

1. Confirm the exact list of P1 features across all roles (from this document) so your demo script is scoped to what will *actually* exist, not what's aspirational.
2. Start drafting the demo script now, even against mocked/fake data – refine it as real features land instead of writing it cold on the last day.

7 48-Hour Master Timeline

Time	Milestone (whole team)
Hour 0–4	Everyone sets up their own tooling in parallel. Hardware: single sensor reading over serial. Mobile: fake BLE service connects to app. DB: schema agreed and stubbed. AI: fake-data signal pipeline runs. Frontend: demo script skeleton drafted.
Hour 4–12	First real end-to-end data flow: real (even if noisy) ESP32 data → phone → dashboard showing <i>something</i> live. All P1 features across roles actively in progress.
Hour 12–24	P1 features across every role complete and integrated together. First full run-through of the demo script, even if rough.
Hour 24–36	P2 features layered in: alerts firing off real logic, SOS working, personalization baseline running, dashboard visuals polished to match mockups.
Hour 36–44	Stop building new features. Focus entirely on integration bugs, edge cases in the demo path, and rehearsing the pitch. Record the backup demo video now.
Hour 44–48	Final rehearsal, slides finalized, everyone sleeps at least a little before presenting.

Final Word

Every role in this document has been scoped to what is genuinely buildable by one or two people in 48 hours, using well-established, real techniques (Pan-Tompkins-style R-peak detection, standard heat-index formulas, threshold-based fall detection, rolling-baseline personalization) rather than anything that requires training data or weeks of tuning. If a judge asks “is this really AI,” the honest and correct answer is: *“it’s signal-processing-driven risk detection with adaptive personal baselines, running fully on-device – with trained deep learning models as an explicit next step,”* and that is a strong, technically credible answer.